

Recompensas DAG con *Achievement Triggers* para un Humanoide

Adaptación del paper *From Rolling Over to Walking* (Meng & Xiao, 2023) a hardware de consumo con PPO y MuJoCo

Robótica Cognitiva · Universidad Autónoma del Estado de México (UAEM)

Alan Cabrera y Enrique Albarrán · RTX 4050 Laptop · junio 2026

1. Introducción y objetivo

Entrenar a un robot humanoide a incorporarse y caminar desde una posición tendida es un problema clásico de control con recompensa dispersa: la señal de éxito (estar de pie) está muy lejos del estado inicial, y una recompensa monolítica rara vez guía al agente hasta ahí. El paper de Meng & Xiao (2023) propone descomponer la tarea en un **grafo dirigido acíclico (DAG) de habilidades** encadenadas mediante *achievement triggers*. Este proyecto implementa esa idea central a una escala entrenable en una laptop con GPU de consumo, y documenta de forma honesta hasta dónde llega el método con ese presupuesto.

El objetivo no es reproducir el paper al pie de la letra, sino (a) implementar su mecanismo de recompensa, (b) entrenarlo sobre HumanoidStandup-v4 (17 grados de libertad, MuJoCo) con PPO, y (c) analizar rigurosamente el comportamiento resultante.

2. La idea del paper en una fórmula

En lugar de una sola recompensa, se definen habilidades (p. ej. *lift* → *standup* → *upright*) donde el bono de una habilidad **solo se activa** cuando la anterior supera un umbral (*passing score*, PS). El "logro" (*achievement*) acumula el mejor avance histórico:

$$A_{ij,t} = \max(A_{ij,t-1}, R_{i,t} - PS_{ij}) \quad R_t = \sum_i \sum_j A_{ij,t} \cdot R_{j,t}$$

Así, la recompensa de la habilidad *j* solo contribuye al total una vez que su habilidad padre *i* ha sido "alcanzada", encadenando el aprendizaje de lo simple a lo complejo y evitando el problema de la recompensa dispersa.

3. Adaptaciones por limitación de hardware

Paper original	Esta implementación
Isaac Gym, miles de envs en GPU	MuJoCo en CPU, hasta 10 envs en paralelo
Robot iCub (32 DoF)	HumanoidStandup-v4 (17 DoF)
Decenas de millones de steps	3–6 millones de steps
DAG completo hasta caminar	DAG de incorporación (<i>lift</i> → <i>standup</i> → <i>upright</i>)

Componentes del paper implementados además del DAG: **señales CPG** (8 senos/cosenos de frecuencias coprimas concatenados a la observación), **action clamping** proporcional al progreso del episodio, y **representación ego-céntrica** (velocidades rotadas al marco de referencia de la raíz del robot, verificada contra `mj_objectVelocity`).

4. Metodología y proceso experimental

El sistema se construyó como un *wrapper* de Gymnasium que (i) calcula la recompensa de cada habilidad a partir del estado físico de MuJoCo, (ii) aplica las fórmulas del DAG, y (iii) añade CPG y transformación ego-céntrica a la observación. Se entrena con PPO (stable-baselines3, red [400,300,200,100]). El proyecto avanzó por iteraciones, y el mayor valor está en el **proceso de depuración**: cada fallo reveló un principio de diseño de recompensas.

Iter.	Qué ocurrió	Lección
v1	Passing score inalcanzable: el bono nunca se activaba	Calibrar PS al rango real de la señal
v2	Reward de altura "ciego": $\text{clip}(z-0.30)$ tiene gradiente 0 en $z \in [0.08, 0.15]$, donde el robot opera	Una señal densa puede ser sparse de facto
v3	FIX: rampa de altura continua desde el suelo $\rightarrow$ el robot se incorpora y se sienta erguido	Medir la cantidad física (z), no solo métricas de PPO
v4	feet_tuck + warm-start no escapa la pose en "L"	Un bono débil no saca de un óptimo local estable

El bug de la iteración v2 fue el hallazgo central. Midiendo la altura real del torso z del modelo entrenado se descubrió que el robot vivía en $z \in [0.076, 0.152]$, pero las tres señales de altura usaban umbrales ($z > 0.30$, > 0.80 , > 1.00) que valen exactamente cero —y con gradiente cero— en todo ese rango. Las señales de altura estaban **muertas**: el agente solo podía optimizar las señales de ángulo, y las explotaba (*reward hacking*). La corrección fue reemplazar los umbrales por **rampas continuas** con gradiente no nulo desde el suelo, de modo que cada centímetro ganado en altura diera más recompensa.



Figura 1. Comportamiento antes (v2, recompensa de altura ciega) y después (v3, rampa continua) del fix. v2 se retuerce en el piso; v3 se incorpora.

5. Resultados

La corrección del reward (v3) produjo un cambio cualitativo: el robot pasó de retorcerse acostado a hacer un esfuerzo coordinado de incorporarse hasta sentarse erguido. La iteración v4, que intentó llevarlo de sentado a de pie añadiendo una señal de "recoger los pies", no lo logró: la pose de "sentado en L" resultó ser un óptimo local demasiado estable.

Métrica	v2 (reward roto)	v3 (rampa altura)	v4 (feet_tuck)
Recompensa media por episodio	~67 (plana)	~850 (sube)	~900
Altura z máxima	0.152	0.450	0.429
Altura z promedio	0.116	0.291	0.209
% del tiempo despegado ($z > 0.30$)	0 %	42 %	—
% del tiempo de pie ($z > 0.80$)	0 %	0 %	0 %
std de PPO (estabilidad)	4.4 (explota)	0.94	0.93

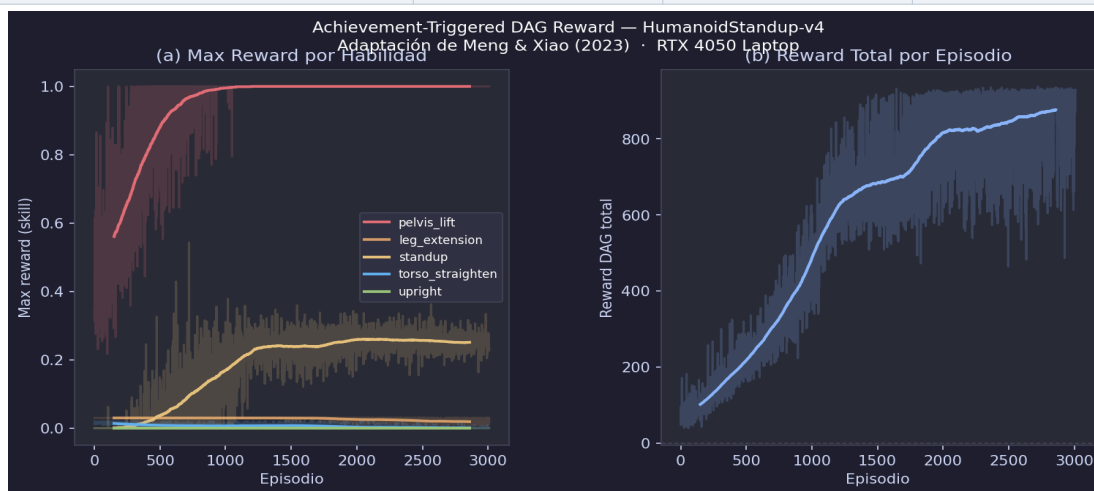


Figura 2. Curvas de v3. Izq.: recompensa máxima por habilidad — pelvis_lift satura en 1.0 y standup despegue de 0. Der.: recompensa total por episodio, crecimiento sostenido de ~100 a ~870.

6. Hallazgos y lecciones

(1) Sparse de facto: una recompensa "densa" no guía si su gradiente está fuera de la región donde el agente opera. **(2) Reward hacking:** el agente explota sistemáticamente la señal más fácil (ángulos, pose en L) antes que el objetivo. **(3) Proxy $\neq$ objetivo:** en v4 la recompensa total subió mientras la altura real bajó. **(4) Escapar un óptimo local estable** requiere más que un bono positivo débil: penalización activa, mayor exploración o un curriculum.

7. Limitaciones y conclusión

Limitaciones: una sola semilla por configuración (sin validación estadística); presupuesto de muestras 2–3 órdenes de magnitud menor que el del paper; robot y simulador distintos; el robot no llega a pararse del todo —un límite del presupuesto de cómputo, no del método—.

Conclusión: se implementó la contribución central del paper (recompensa DAG con *achievement triggers*, junto con CPG, *action clamping* y representación ego-céntrica) y se llevó al humanoide de *tendido* a *sentado erguido* en hardware de consumo. Más allá del resultado, el proyecto aporta un proceso de diagnóstico riguroso que expone errores comunes (y sutiles) del diseño de recompensas en aprendizaje por refuerzo.

Referencia

Meng, Y., & Xiao, J. (2023). *From Rolling Over to Walking: Enabling Humanoid Robots to Develop Complex Motor Skills*. arXiv:2303.02581. Código del proyecto: github.com/AlanoHater/robot-ppo (rama feature/multipath-dag-egocentric).